Sperimentando con diverse decomposizioni di una formula logica

Davide Camino

November 11, 2026

Abstract

In questi esperimenti trasformiamo una formula logica prima in un circuito digitale, poi in un problema di ottimizzazione infine tentiamo di trovare una soluzione al problema di ottimizzazione con vari metodi. Lo scopo degli esperimenti è valutare se diverse traduzioni in circuito digitale (usando set di porte logiche diverse) o diverse traduzioni in problemi di ottimizzazione (usando diverse forme energetiche a parita di gate) impattano la qualità delle soluzioni del problema di ottimizzazione.

Nella Sezione 1 vengono presentati la configurazione sperimentale, la formula logica di partenza, i diversi circuiti logici e le funzioni di energia adottate. La Sezione 2 riporta i risultati ottenuti e un breve commento. Infine, la Sezione 3 discute i limiti dello studio e propone alcuni spunti per sviluppi futuri.

1 Configurazione sperimentale

1.1 Formula logica

La formula logica di partenza è:

$$Y = C \wedge \neg(A \vee B). \tag{1}$$

La formula è volutamente semplice, così da consentire di individuare facilmente un assegnamento che la soddisfi. È stata inoltre scelta in modo che tale assegnamento sia unico. Di conseguenza, una volta tradotta la formula in un problema di ottimizzazione, lo spazio delle soluzioni presenta un unico minimo globale.

```
module sat (
input  wire A,
input  wire B,
input  wire C,
output wire Y
);

        assign Y = C & ~(A | B);

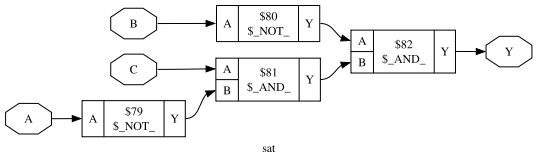
endmodule
```

La formula è stata codificata direttamente in Verilog per mantenere un controllo puntuale sui bit utilizzati e consentirne la trasformazione in diversi circuiti logici mediante *Yosys*. Il listato a destra mostra una possibile codifica della formula in Verilog.

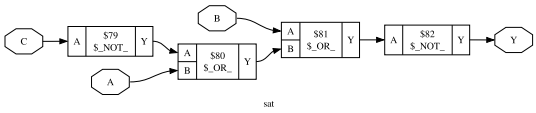
1.2 Circuiti digitali

La compilazione del programma Verilog in un circuito digitale è stata effettuata con Yosys[1]. Yosys è un software di sintesi hardware articolato e ricco di funzionalità, che esegue diverse

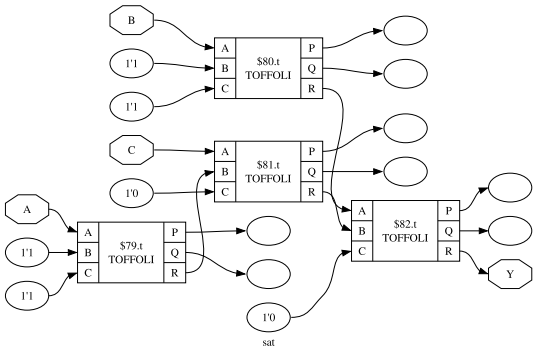
[1] https://yosyshq.net/yosys/



(a) Sole porte `NOT` e `AND`



(b) Sole porte `NOT` e `OR`



(c) Sole porte `TOFFOLI`

Figure 1: Circuiti esaminati

fasi di compilazione e semplificazione del circuito. Le opzioni di compilazione possono essere specificate in un file di configurazione, solitamente denominato `synth.ys`.

Per realizzare gli esperimenti sono stati utilizzati due componenti del processo di compilazione di Yosys:

- ABC[2]: un insieme di strumenti per l'ottimizzazione logica a grana fine e il mapping su LUT. In questi esperimenti ABC viene utilizzato per stabilire quali porte siano ammesse nel circuito digitale finale. Ad esempio, aggiungendo al file `synth.ys` il comando `abc -g AND,XOR`, le sole porte logiche che possono comparire nel circuito finale sono `AND`, `XOR` e `NOT` (quest'ultima è sempre inclusa);
- Techmap: una fase di mapping tecnologico che sostituisce le celle del progetto con implementazioni specificate in un file sorgente Verilog. In questo caso Techmap viene utilizzato per definire porte logiche personalizzate, come la porta `TOFFOLI`. Dopo l'esecuzione di ABC, viene definita una corrispondenza per convertire le porte generate nelle porte desiderate.

Sono stati esaminati tre circuiti con diverse combinazioni di porte logiche, mostrati nella Figura 1.

1.3 Funzioni di energia

A ogni porta logica è associata una funzione pseudobooleana che assegna un'energia a ciascuna configurazione dei bit in ingresso e in uscita. Tutte le configurazioni compatibili con la tavola di verità della porta corrispondono a minimi globali dell'energia. Questo consente di tradurre il vincolo imposto dalla porta logica in un problema di minimizzazione. La funzione di energia è espressa come un polinomio; la funzione pseudobooleana che rappresenta l'intero circuito si ottiene sommando i polinomi associati alle singole porte.

Di seguito sono riportati i polinomi associati a ciascuna porta. Poiché il problema è formulato in forma Ising, ogni variabile soddisfa $var \in \{+1, -1\}$; per convenzione, $+1$ rappresenta il valore di verità `True` e -1 il valore `False`. Le lettere maiuscole indicano i valori dei bit in ingresso (A, B, C) e in uscita (Y) dalle porte, mentre le lettere minuscole indicano variabili ancillari. I polinomi 4 e 5 definiscono due versioni della porta `AND`, rispettivamente senza e con variabili

[2] https://github.com/berkeley-abc/abc

ancillari. Nei risultati vengono quindi presentate e discusse due formulazioni dello stesso circuito digitale, ottenute utilizzando le due versioni della porta.

L'introduzione di una variabile ancillare nella formulazione della porta `AND` mira ad aumentare la separazione tra il minimo globale della funzione pseudobooleana e il livello energetico immediatamente superiore. Nel problema QUBO o Ising, ciò corrisponde ad ampliare il divario energetico tra lo stato ground e il primo stato eccitato.

La motivazione teorica di questa scelta risiede nel teorema adiabatico, che impone vincoli sul tempo di annealing legati al divario energetico tra lo stato ground e il primo stato eccitato: al diminuire di tale divario, aumentano i tempi richiesti per mantenere l'evoluzione adiabatica.

$$\texttt{NOT}: 3AY \tag{2}$$
$$\texttt{OR}: +A+B-2Y+AB-2AY-2BY \tag{3}$$
$$\texttt{AND}: -A-B+2Y+AB-2AY-2BY \tag{4}$$
$$\texttt{AND-ancillary}: -A-B+Y+x+AB-AY-Ax-BY-Bx-Yx \tag{5}$$
$$\texttt{TOFFOLI}: -A-B-C-Y+3x+2y+AB-2Ax-2Bx+CY-Cx-2Cy-Yx-2Yy+2xy \tag{6}$$

1.4 Esecuzione degli esperimenti

Per eseguire gli esperimenti sono stati sviluppati la libreria Python *OpenQUOPS*[3]; alcune utility[4] estrarre dai file generati da Yosys una lista di gates, e per trasformare questa lista di porte logiche prima in un polinomio e successivamente in un problema in forma Ising; infine uno script[5] per automatizzare l'esecuzione delle prove. Gli esperimenti sono stati eseguiti su Fedarchs[6].

OpenQUOPS (Open Quadratic Unconstrained Optimization Problem Solver) consente di definire e manipolare problemi di ottimizzazione in forma QUBO o Ising. La libreria si interfaccia con la libreria Python *qubovert*[7], permettendo di importare problemi definiti in qubovert e di esportare quelli definiti in OpenQUOPS. OpenQUOPS integra inoltre diversi solver per problemi QUBO o Ising. Al momento sono disponibili la ricerca bruteforce, il simulated annealing, il simulated quantum annealing e il QAOA simulato.

I test hanno riguardato tutti i metodi probabilistici disponibili in OpenQUOPS. Per ciascun metodo e ciascun circuito sono state eseguite 30 prove indipendenti, calcolando poi la media e la deviazione standard dei risultati raccolti. Ogni prova prevedeva 2^{16} samples. Per QAOA è stato costruito un circuito a 2 livelli ed è stata adottata una policy di restarting dell'ottimizzazione dei parametri: ogni run prevede 20 ottimizzazioni indipendenti, tra le quali viene selezionata quella che consente di ottenere l'energia più bassa.

2 Risultati

I risultati sono riportati nella Figura 2, organizzata come una tabella: le colonne corrispondono ai metodi di risoluzione e le righe ai diversi circuiti. In ciascun grafico della Figura 2, il secondo punto rappresenta il numero di osservazioni dell'assegnamento corretto ed è evidenziato in grassetto rispetto agli altri. La disponibilità di numerose porte logiche rende pressoché illimitate le combinazioni con cui decomporre una formula. Tuttavia, già le quattro decomposizioni considerate consentono di formulare alcune osservazioni:

[3] https://github.com/DavideCamino/OpenQUOPS
[4] https://github.com/DavideCamino/Reports/tree/main/Gates_to_Ising/code/utility
[5] https://github.com/DavideCamino/Reports/tree/main/Gates_to_Ising/code
[6] FEDARCHS cluster running on OpenStack available to α group & friends.
[7] https://qubovert.readthedocs.io/en/latest/index.html

- I risultati ottenuti con QAOA seguono un andamento simile a quelli del simulated quantum annealing, ma presentano un rumore molto maggiore;
- In assenza di variabili ancillari, e quindi a parità di numero di variabili utilizzate, la combinazione `AND-NOT` sembra fornire risultati migliori della combinazione `OR-NOT`;
- A parità di porte logiche, l'impiego di polinomi senza variabili ancillari consente di ottenere risultati di qualità superiore. In questo caso, l'aumento del numero di variabili sembra introdurre più rumore di quanto se ne riesca a ridurre aumentando la separazione energetica tra lo stato ground e il primo stato eccitato;
- Probabilmente per ragioni analoghe, il caso caratterizzato dal maggior rumore è quello del circuito composto da sole porte `TOFFOLI`, che presenta anche il maggior numero di variabili.

3 Limitazioni

Lo studio presenta le seguenti limitazioni, che suggeriscono possibili direzioni di approfondimento:

- Le combinazioni di porte logiche esaminate sono molto poche;
- Manca una definizione della porta `AND` con due variabili ancillari;
- Anche la porta `OR` può essere formulata mediante un polinomio con una o due variabili ancillari, ma queste formulazioni non sono state considerate;
- Non è stata condotta un'analisi sistematica di tutte le porte per individuare e selezionare quelle che forniscono i risultati migliori;
- Quando l'insieme delle porte è generato da Yosys utilizzando solo ABC, il circuito è ottimizzato per usare meno porte possibili. Quando invece si ricorre a Techmap, la conversione in porte `TOFFOLI` avviene mediante una sostituzione uno a uno, senza considerare possibili raggruppamenti. Ad esempio, una porta `AND` seguita da una porta `XOR` potrebbe essere sostituita da un'unica porta `TOFFOLI` che ne realizzi la funzione complessiva; Techmap genera invece due porte, ciascuna delle quali implementa una singola porta del circuito originale.

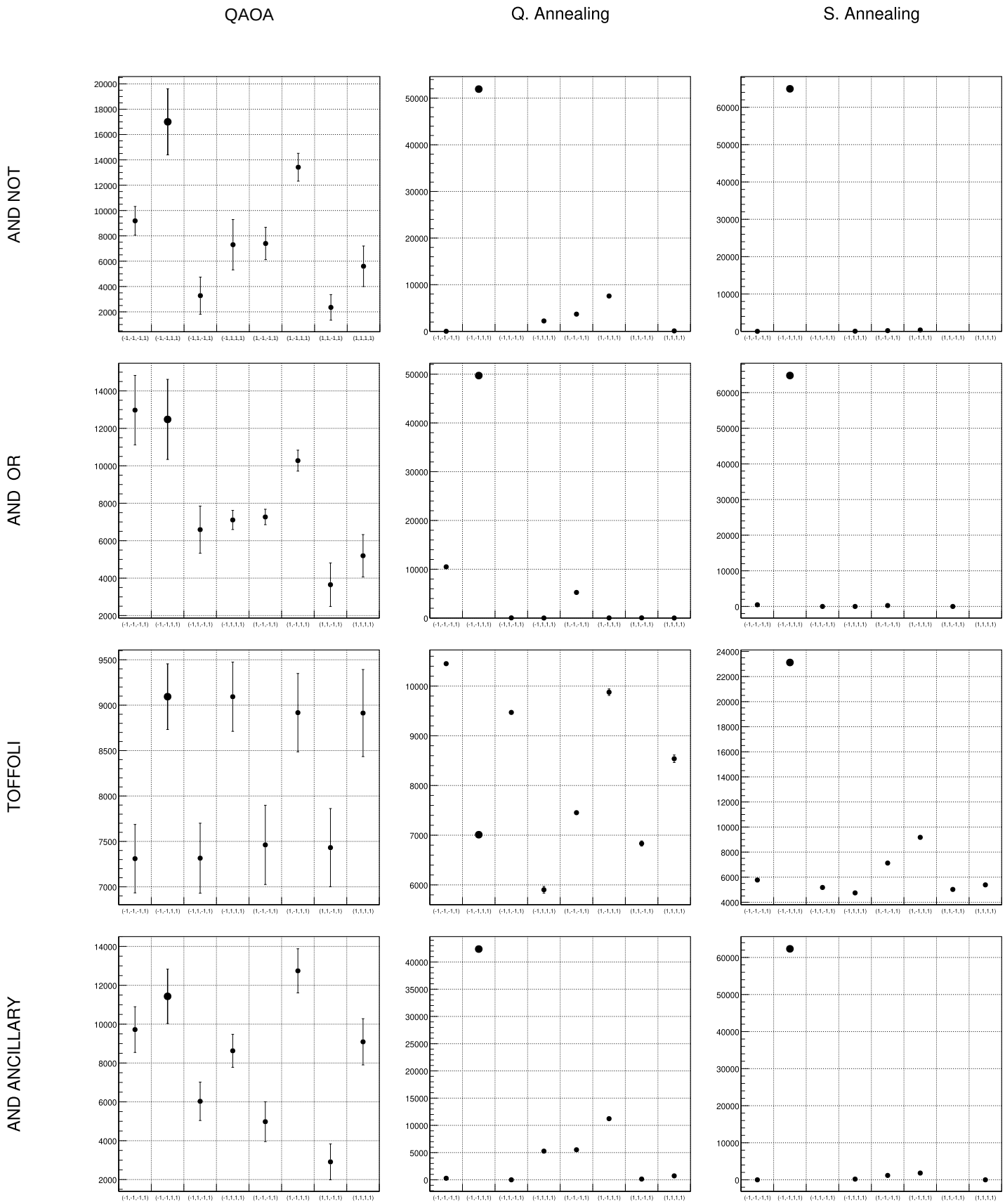


Figure 2: Risultati